VILORA

BACKEND API DOCUMENTATION

API Specification & Team Integration Blueprint

Version	v1.0
Base URL	http://localhost:8080
Format	JSON
Frontend	React / Vite
Backend	Spring Boot
Database	Relational DB / JPA

Purpose

This document is the shared API contract for the two-member Vilora backend team. It is designed so backend developers and the React frontend can work against the same endpoint names, request shapes, response shapes, authentication rules, and implementation order.

Documentation style follows the practical endpoint-by-endpoint structure used in the team's G2 API reference: endpoint name, purpose, HTTP type, request body, and success response, with additional authentication and error guidance for Vilora.

1. API OVERVIEW

VILORA BACKEND FLOW

```
React Frontend
      |
      | HTTP / JSON
      v
Spring Boot REST API
      |
      +--> Controller
      +--> Service
      +--> Repository
      v
Relational Database
```

Base URL: http://localhost:8080

Content-Type: application/json

Authentication: protected endpoints use **Authorization: Bearer <JWT>**.

Response convention: Vilora follows a simple envelope with **statusCode**, optional **data**, and optional **message**.

Example common response

```
{
  "statusCode": 200,
  "data": { }
}
```

Example common error

```
{
  "statusCode": 404,
  "message": "Product not found"
}
```

HTTP status conventions

Status	Meaning	Typical use
200	OK	Successful GET / PUT / PATCH
201	Created	Successful POST creating a resource
204	No Content	Successful DELETE
400	Bad Request	Validation / invalid parameters
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Authenticated but not allowed
404	Not Found	Resource does not exist
409	Conflict	Stock / duplicate / state conflict
500	Server Error	Unexpected backend failure

2. API INDEX

Module	Methods	Endpoints
AUTH	GET, POST	3
CATEGORIES	DELETE, GET, POST, PUT	5
PRODUCTS	DELETE, GET, POST, PUT	5
CART	DELETE, GET, PATCH, POST	5
WISHLIST	DELETE, GET, POST	3
ADDRESSES	DELETE, GET, POST, PUT	5
ORDERS	GET, PATCH, POST	4
PAYMENTS	GET, POST	3
USERS	GET, PUT	2
REVIEWS	DELETE, GET, POST, PUT	4

Recommended implementation sequence

```
1. Categories
2. Products
3. Product search / filter / sort / pagination
4. Authentication
5. Cart
6. Addresses
7. Orders
8. Wishlist
9. Payments
10. Reviews
11. User profile / account integration
```

The endpoint list is intentionally defined before implementation. The two backend developers can divide modules while keeping the endpoint contract unchanged.

AUTH

Authentication establishes the user identity used by protected cart, address, order, wishlist, review, payment, and profile APIs.

1. /api/auth/register

POST **REGISTER A NEW CUSTOMER**

AUTH: Public

REQUEST BODY

```
{
  "name": "Ananya Mehta",
  "email": "ananya@example.com",
  "password": "StrongPassword123"
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "userId": 1,
    "name": "Ananya Mehta",
    "email": "ananya@example.com"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 409,
  "message": "Email already registered"
}
```

NOTE: Password must be stored as a secure hash; never return the password.

2. /api/auth/login

POST **LOGIN A CUSTOMER**

AUTH: Public

REQUEST BODY

```
{
  "email": "ananya@example.com",
  "password": "StrongPassword123"
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "user": {
      "id": 1,
      "name": "Ananya Mehta",
      "email": "ananya@example.com"
    },
    "accessToken": "jwt-access-token",
    "refreshToken": "refresh-token"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 401,
  "message": "Invalid email or password"
}
```

NOTE: Frontend can store the returned authentication state in its auth store.

3. /api/auth/me

GET **FETCH THE CURRENT LOGGED-IN USER**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "name": "Ananya Mehta",
    "email": "ananya@example.com"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 401,
  "message": "Unauthorized"
}
```

CATEGORIES

Category APIs provide the category list used by Shop filters and product classification.

4. /api/categories

GET **GET ACTIVE PRODUCT CATEGORIES**

AUTH: Public

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": [
    {"id": 1, "name": "Dry Fruits", "description": "Premium nuts and dried fruits"},
    {"id": 2, "name": "Herbs & Spices", "description": "Natural herbs and spices"}
  ]
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 200,
  "data": []
}
```

5. /api/categories/{id}

GET **GET A CATEGORY BY ID**

AUTH: Public

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "name": "Dry Fruits",
    "description": "Premium nuts and dried fruits"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Category not found"
}
```

6. /api/categories

POST **CREATE A CATEGORY**

AUTH: Admin JWT

REQUEST BODY

```
{
  "name": "Dry Fruits",
  "description": "Premium nuts and dried fruits",
  "imageUrl": "https://example.com/dry-fruits.jpg"
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "id": 1,
    "name": "Dry Fruits"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 409,
  "message": "Category already exists"
}
```

7. /api/categories/{id}

PUT **UPDATE A CATEGORY**

AUTH: Admin JWT

REQUEST BODY

```
{
  "name": "Dry Fruits",
  "description": "Premium quality dry fruits",
  "imageUrl": "https://example.com/dry-fruits.jpg"
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "name": "Dry Fruits",
    "description": "Premium quality dry fruits"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Category not found"
}
```

8. /api/categories/{id}

DELETE **DELETE / DEACTIVATE A CATEGORY**

AUTH: Admin JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 204
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Category not found"
}
```

NOTE: Prefer soft-delete/deactivation if products still reference the category.

PRODUCTS

Product APIs drive the Shop page, product cards, search, filtering, sorting, pagination, and product detail screens.

9. /api/products

GET **GET PRODUCTS WITH SEARCH, FILTER, SORT AND PAGINATION**

AUTH: Public

REQUEST BODY

```
Query parameters:
search        optional string
categoryId    optional long
minPrice      optional decimal
maxPrice      optional decimal
minRating     optional decimal
inStock       optional boolean
sort          optional: relevance|price_asc|price_desc|rating
page          optional integer, default 0
size          optional integer, default 12
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "content": [
      {
        "id": 1,
        "name": "Himalayan Almonds",
        "category": "Dry Fruits",
        "price": 649,
        "originalPrice": 799,
        "discount": 19,
        "rating": 4.8,
        "reviewCount": 124,
        "image": "https://example.com/almonds.jpg",
        "isOrganic": true,
        "inStock": true
      }
    ],
    "page": 0,
    "size": 12,
    "totalElements": 18,
    "totalPages": 2
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 400,
  "message": "Invalid product filter"
}
```

NOTE: One endpoint handles the Shop page filters shown in the React frontend.

10. /api/products/{id}

GET **GET PRODUCT DETAILS**

AUTH: Public

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "name": "Himalayan Almonds",
    "description": "Carefully sourced premium almonds.",
    "category": "Dry Fruits",
    "price": 649,
    "originalPrice": 799,
    "discount": 19,
    "stockQuantity": 25,
    "rating": 4.8,
    "reviewCount": 124,
    "image": "https://example.com/almonds.jpg",
    "isOrganic": true,
    "inStock": true
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Product not found"
}
```

11. /api/products

POST **CREATE A PRODUCT**

AUTH: Admin JWT

REQUEST BODY

```
{
  "name": "Himalayan Almonds",
  "description": "Carefully sourced premium almonds.",
  "categoryId": 1,
  "price": 649,
  "originalPrice": 799,
  "stockQuantity": 25,
  "imageUrl": "https://example.com/almonds.jpg",
  "organic": true
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "id": 1,
    "name": "Himalayan Almonds"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 400,
  "message": "Invalid product data"
}
```

12. /api/products/{id}

PUT **UPDATE A PRODUCT**

AUTH: Admin JWT

REQUEST BODY

```
{
  "name": "Himalayan Almonds",
  "description": "Updated product description.",
  "categoryId": 1,
  "price": 699,
  "originalPrice": 799,
  "stockQuantity": 40,
  "imageUrl": "https://example.com/almonds.jpg",
  "organic": true
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "name": "Himalayan Almonds",
    "price": 699
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Product not found"
}
```

13. /api/products/{id}

DELETE **DELETE / DEACTIVATE A PRODUCT**

AUTH: Admin JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 204
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Product not found"
}
```

NOTE: Prefer marking products inactive rather than physically deleting historical order references.

CART

Cart APIs replace the frontend's local cart persistence with server-side cart state for authenticated users.

14. /api/cart

GET **GET THE CURRENT USER CART**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "items": [
      {
        "id": 11,
        "productId": 1,
        "productName": "Himalayan Almonds",
        "price": 649,
        "quantity": 1,
        "lineTotal": 649,
        "image": "https://example.com/almonds.jpg"
      }
    ],
    "subtotal": 649,
    "shipping": 0,
    "tax": 32,
    "total": 681
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 200,
  "data": {
    "items": [],
    "subtotal": 0,
    "shipping": 0,
    "tax": 0,
    "total": 0
  }
}
```

15. /api/cart/items

POST **ADD A PRODUCT TO CART**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "productId": 1,
  "quantity": 1
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "cartItemId": 11,
    "productId": 1,
    "quantity": 1,
    "lineTotal": 649
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 409,
  "message": "Insufficient stock"
}
```

16. /api/cart/items/{id}

PATCH **UPDATE CART ITEM QUANTITY**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "quantity": 2
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "cartItemId": 11,
    "quantity": 2,
    "lineTotal": 1298
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 400,
  "message": "Quantity must be greater than zero"
}
```

17. /api/cart/items/{id}

DELETE **REMOVE A CART ITEM**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 204
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Cart item not found"
}
```

18. /api/cart

DELETE **CLEAR THE CURRENT USER CART**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 204
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Cart not found"
}
```

WISHLIST

Wishlist APIs support the heart/favourite controls used by product cards and the account wishlist page.

19. /api/wishlist

GET **GET THE CURRENT USER WISHLIST**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": [
    {
      "productId": 1,
      "name": "Himalayan Almonds",
      "price": 649,
      "image": "https://example.com/almonds.jpg"
    }
  ]
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 200,
  "data": []
}
```

20. /api/wishlist/{productId}

POST **ADD A PRODUCT TO WISHLIST**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "productId": 1,
    "added": true
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Product not found"
}
```

21. /api/wishlist/{productId}

DELETE **REMOVE A PRODUCT FROM WISHLIST**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 204
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Wishlist item not found"
}
```

ADDRESSES

Address APIs support the Delivery Address step shown in checkout, including multiple saved addresses and a default address.

22. /api/addresses

GET **GET SAVED DELIVERY ADDRESSES**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": [
    {
      "id": 1,
      "fullName": "Ananya Mehta",
      "phone": "+91 98765 43210",
      "addressLine1": "24 Green Park",
      "addressLine2": "Andheri West",
      "city": "Mumbai",
      "state": "Maharashtra",
      "pincode": "400053",
      "isDefault": true
    }
  ]
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 200,
  "data": []
}
```

23. /api/addresses/{id}

GET **GET ONE SAVED ADDRESS**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "fullName": "Ananya Mehta",
    "phone": "+91 98765 43210",
    "addressLine1": "24 Green Park",
    "city": "Mumbai",
    "state": "Maharashtra",
    "pincode": "400053",
    "isDefault": true
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Address not found"
}
```

24. /api/addresses

POST **ADD A DELIVERY ADDRESS**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "fullName": "Ananya Mehta",
  "phone": "+91 98765 43210",
  "addressLine1": "24 Green Park",
  "addressLine2": "Andheri West",
  "city": "Mumbai",
  "state": "Maharashtra",
  "pincode": "400053",
  "isDefault": true
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "id": 1,
    "fullName": "Ananya Mehta",
    "isDefault": true
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 400,
  "message": "Invalid address data"
}
```

25. /api/addresses/{id}

PUT **UPDATE A DELIVERY ADDRESS**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "fullName": "Ananya Mehta",
  "phone": "+91 98765 43210",
  "addressLine1": "24 Green Park",
  "addressLine2": "Andheri West",
  "city": "Mumbai",
  "state": "Maharashtra",
  "pincode": "400053",
  "isDefault": true
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "fullName": "Ananya Mehta"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Address not found"
}
```

26. /api/addresses/{id}

DELETE **DELETE A DELIVERY ADDRESS**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 204
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Address not found"
}
```

ORDERS

Order APIs convert the checkout cart into an immutable order snapshot and provide order history/detail/cancellation.

27. /api/orders

POST **CREATE AN ORDER FROM THE CURRENT CART**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "addressId": 1,
  "paymentMethod": "COD"
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "orderId": 1001,
    "status": "PENDING",
    "subtotal": 649,
    "shipping": 0,
    "tax": 32,
    "total": 681
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 409,
  "message": "Cart is empty or stock changed"
}
```

NOTE: Order creation must snapshot product price/name so historical orders do not change when products are edited.

28. /api/orders

GET **GET CURRENT USER ORDER HISTORY**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": [
    {
      "id": 1001,
      "status": "PENDING",
      "total": 681,
      "createdAt": "2026-09-03T10:30:00Z"
    }
  ]
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 200,
  "data": []
}
```

29. /api/orders/{id}

GET **GET ORDER DETAILS**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1001,
    "status": "PENDING",
    "items": [
      {
        "productId": 1,
        "productName": "Himalayan Almonds",
        "quantity": 1,
        "unitPrice": 649,
        "lineTotal": 649
      }
    ],
    "address": {
      "city": "Mumbai",
      "pincode": "400053"
    },
    "total": 681
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Order not found"
}
```

30. /api/orders/{id}/cancel

PATCH **CANCEL AN ELIGIBLE ORDER**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "orderId": 1001,
    "status": "CANCELLED"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 409,
  "message": "Order cannot be cancelled at its current status"
}
```

PAYMENTS

Payment APIs are intentionally separated from order creation so a payment gateway can be integrated without changing the frontend contract.

31. /api/payments/create

POST **CREATE A PAYMENT REQUEST**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "orderId": 1001,
  "paymentMethod": "ONLINE"
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "paymentId": 501,
    "orderId": 1001,
    "status": "CREATED",
    "amount": 681,
    "currency": "INR"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 400,
  "message": "Invalid payment request"
}
```

NOTE: Payment gateway integration details will be added when the gateway is selected.

32. /api/payments/verify

POST VERIFY A PAYMENT CALLBACK / RESULT

AUTH: Bearer JWT

REQUEST BODY

```
{
  "paymentId": 501,
  "gatewayPaymentId": "gateway-payment-id",
  "signature": "gateway-signature"
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "paymentId": 501,
    "status": "SUCCESS"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 400,
  "message": "Payment verification failed"
}
```

NOTE: Never trust a client-only success flag; verify the gateway signature/server response.

33. /api/payments/{id}

GET GET PAYMENT STATUS

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "paymentId": 501,
    "orderId": 1001,
    "status": "SUCCESS",
    "amount": 681,
    "currency": "INR"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Payment not found"
}
```

USERS

User APIs support the Account/Profile page after authentication.

34. /api/users/me

GET **GET CURRENT USER PROFILE**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "name": "Ananya Mehta",
    "email": "ananya@example.com",
    "phone": "+91 98765 43210"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 401,
  "message": "Unauthorized"
}
```

35. /api/users/me

PUT **UPDATE CURRENT USER PROFILE**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "name": "Ananya Mehta",
  "phone": "+91 98765 43210"
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 1,
    "name": "Ananya Mehta",
    "phone": "+91 98765 43210"
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 400,
  "message": "Invalid profile data"
}
```

REVIEWS

Review APIs provide product ratings and comments and can later drive the product rating aggregates.

36. /api/products/{productId}/reviews

GET **GET REVIEWS FOR A PRODUCT**

AUTH: Public

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "content": [
      {
        "id": 10,
        "userName": "Ananya",
        "rating": 5,
        "comment": "Fresh and crunchy.",
        "createdAt": "2026-09-01T10:00:00Z"
      }
    ],
    "page": 0,
    "size": 10,
    "totalElements": 24
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 404,
  "message": "Product not found"
}
```

37. /api/products/{productId}/reviews

POST **SUBMIT A PRODUCT REVIEW**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "rating": 5,
  "comment": "Fresh and crunchy."
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 201,
  "data": {
    "id": 10,
    "productId": 1,
    "rating": 5,
    "comment": "Fresh and crunchy."
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 409,
  "message": "Review already submitted for this order item"
}
```

NOTE: Recommended rule: only customers who purchased the product can submit a review.

38. /api/reviews/{id}

PUT **UPDATE A REVIEW**

AUTH: Bearer JWT

REQUEST BODY

```
{
  "rating": 4,
  "comment": "Still very good."
}
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 200,
  "data": {
    "id": 10,
    "rating": 4,
    "comment": "Still very good."
  }
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 403,
  "message": "You cannot edit this review"
}
```

39. /api/reviews/{id}

DELETE **DELETE A REVIEW**

AUTH: Bearer JWT

REQUEST BODY

```
NONE
```

ON SUCCESS RESPONSE

```
{
  "statusCode": 204
}
```

COMMON ERROR RESPONSES

```
{
  "statusCode": 403,
  "message": "You cannot delete this review"
}
```

3. BACKEND DESIGN RULES

Recommended Spring Boot package structure

```
com.vilora.backend
|
+-- controller
+-- service
+-- repository
+-- entity
+-- dto
+-- exception
+-- security
+-- config
```

Feature ownership model

Two developers should work by complete feature modules rather than both editing the same Controller/Service files. Each module should be completed and tested before the next dependency-heavy module is started.

Stage	Definition of Done
1. Contract	Endpoint, method, request and response agreed.
2. Entity	Database model and relationships created.
3. Repository	Required queries implemented.
4. Service	Business rules and validation implemented.
5. Controller	HTTP endpoint exposed with DTOs.
6. Test	Postman / integration tests pass.
7. Frontend	React calls the endpoint successfully.
8. Commit	Changes committed with a clear feature name.

Core domain relationships

```
User
 |-- Cart
 |    `-- CartItem --> Product --> Category
 |
 |-- Wishlist --> Product
 |
 |-- Address
 |
 `-- Order
      |-- OrderItem --> Product
      `-- Payment

Product --> Category
Product --> Review <-- User
```

Important persistence rule: OrderItem should keep a snapshot of product name and unit price at purchase time. Product edits must not rewrite historical orders.

4. FRONTEND ↔ BACKEND INTEGRATION MAP

The uploaded React frontend already exposes the screens and local state that these APIs will replace. The integration should happen feature-by-feature.

Frontend area	Primary API	Integration result
Shop / ProductCard	GET /api/products	Products come from Spring Boot instead of seed/mock data.
Product Detail	GET /api/products/{id}	Product page loads real product data.
Login / Register	POST /api/auth/*	Authentication moves to backend.
Cart	GET/POST/PATCH/DELETE /api/cart*	Cart becomes server-side and user-specific.
Checkout Address	/api/addresses	Saved addresses load and persist from backend.
Checkout / Orders	/api/orders	Cart becomes a real order.
Wishlist	/api/wishlist*	Heart state persists for the user.
Account	/api/users/me	Profile comes from backend.
Reviews / Rating	/api/products/{id}/reviews	Real reviews and rating data replace mock values.

Development rule

Do not connect all frontend screens at once. Complete one API module, test it independently, then replace that module's frontend mock/local behavior. This keeps debugging isolated.

5. TEAM CHECKLIST

Before marking an endpoint complete, both backend developers should verify:

1. Endpoint path exactly matches this document.
2. HTTP method exactly matches this document.
3. Request validation is implemented.
4. Success response uses the documented JSON shape.
5. Expected 400 / 401 / 403 / 404 / 409 cases are handled.
6. Protected endpoints reject missing/invalid authentication.
7. User-specific data is scoped to the authenticated user.
8. Database relationships are tested.
9. Postman test is saved in the team's collection.
10. Frontend integration is tested after backend tests pass.

Initial milestone

Start implementation with Categories and Products. Do not begin Cart, Orders, or Payments until the product catalog APIs are working in Postman and the React Shop page can consume them.

Document status: API blueprint — implementation may evolve, but endpoint contracts should be changed only after team agreement.